Reflexão e Síntese

CAN Injection e sua interseção com o YES, CARLA CAN

Disciplina	Segurança Cibernética Automotiva
Atividade	Reflexão e síntese sobre CAN Injection
Autor(a)	_____
Data	Outubro de 2026

Como está organizado este documento. A Seção 1 apresenta os fundamentos da rede CAN (para quem vai estudar ou apresentar o tema do zero). A Seção 2 resume o ataque CAN Injection. As Seções 3 a 5 respondem às três reflexões propostas. A Seção 6 descreve um experimento reproduzível, já incluído no repositório, que demonstra o ataque e uma defesa.

Documento de apoio gerado como rascunho de estudo — revise e personalize antes da entrega.

1. Fundamentos da rede CAN

O que é. O CAN (Controller Area Network) é um barramento serial criado pela Bosch nos anos 1980 (padronizado na ISO 11898) para que as diversas unidades eletrônicas de controle (ECUs) de um veículo — motor, freios, portas, faróis, painel — troquem mensagens entre si. Em vez de ligar cada ECU a todas as outras com fios dedicados, todas compartilham um mesmo par de fios, o que reduz drasticamente o cabeamento e o custo.

1.1 Como o barramento funciona

- **Meio compartilhado e em difusão (broadcast).** O barramento é um par trançado (CAN-H e CAN-L) com sinalização diferencial. **Todos os nós escutam todas as mensagens** — não há circuito ponto a ponto entre dois nós.
- **Mensagens têm assunto, não endereço.** Um frame CAN não carrega 'de quem' nem 'para quem'. Ele é identificado por um **CAN-ID** (11 bits no CAN Clássico, 29 no estendido) que indica o **tipo** da mensagem (p.ex. 'rotação do motor'). Cada ECU decide sozinha quais IDs ela consome.
- **Bit dominante x recessivo e arbitragem.** O bit 0 é **dominante** e o bit 1 é **recessivo**: num 'E' cabeado, se qualquer nó escreve 0, o barramento fica em 0. Quando dois nós transmitem ao mesmo tempo, aquele cujo ID tem mais bits dominantes (ID menor = maior prioridade) **vence a arbitragem** sem destruir a mensagem; o perdedor recua e tenta de novo. É assim que o barramento é compartilhado sem colisões destrutivas.
- **Confirmação e erros.** Os receptores confirmam cada frame no bit de **ACK** (impondo dominante). Qualquer nó que detecte um problema pode emitir um **error frame** (seis bits dominantes) e invalidar o frame em curso. Um nó que acumula muitos erros entra em **Bus Off** e para de transmitir.

1.2 Anatomia simplificada de um frame

Campo	Função
SOF	Start of Frame: marca o início da transmissão.
Identificador (ID)	Identifica o tipo da mensagem e define a prioridade na arbitragem.
Controle (DLC)	Informa quantos bytes de dados o frame carrega (0 a 8 no CAN Clássico).

Dados	Até 8 bytes de payload — os valores em si (ex.: posição do acelerador).
CRC	Checksum para detecção de erros de transmissão.
ACK / EOF	Confirmação pelos receptores e fim do frame.

1.3 Por que o CAN é inseguro por projeto

O CAN Clássico foi concebido para um ambiente **fechado e confiável**, com foco em robustez e determinismo — **não em segurança**. Como consequência direta de seu funcionamento:

- **Sem autenticação de origem:** um receptor não tem como saber qual nó enviou um frame — só vê o ID.
- **Sem criptografia nem verificação de integridade contra adversário:** o conteúdo trafega em texto claro.
- **Qualquer nó pode transmitir qualquer ID.** Se um atacante colocar um dispositivo no barramento, ele pode forjar (fazer *spoofing* de) mensagens de qualquer ECU, e os receptores as aceitarão como legítimas.

Para conter isso, veículos modernos dividem a eletrônica em **vários barramentos CAN** (powertrain, conforto, diagnóstico...) interligados por um **gateway** que roteia e pode filtrar mensagens entre eles. Essa é a base do conceito de **defesa de perímetro** — e, como veremos, é justamente o que o CAN Injection explora.

1.4 Como isso se materializa no YES, CARLA CAN

O YES, CARLA CAN aproxima três mundos: o **comportamento veicular** (simulado no CARLA), a **comunicação intraveicular** (uma rede CAN) e a **segurança** (ataques e defesas). Na prática:

- A rede CAN é **virtual**, criada com o módulo `vcan` do kernel Linux: `vcan0` é o barramento principal do veículo e `vcan1` é o barramento do atacante, ligados por uma ponte `can-gw` (emulando o acesso físico do atacante à rede).
- O arquivo **DBC** (`data/carla.dbc`) é o 'dicionário' da rede: define cada CAN-ID, seus sinais e como codificá-los. A biblioteca `python-can` envia e recebe os frames.
- Os comandos e sensores do CARLA (acelerador, freio, direção, portas, luzes, GNSS, IMU...) são **traduzidos em frames CAN**. Alguns IDs relevantes para este trabalho:

CAN-ID	Mensagem	Tipo	Papel na demo
0x600	THROTTLE	periódico	tráfego normal
0x601	BRAKE	periódico	tráfego normal
0x602	STEER	periódico	tráfego normal
0x604	HAND_BRAKE	periódico	alvo do flood (Fase 3)
0x60C	DOORS	sporadic	alvo do spoof (Fases 2 e 4)
0x60D	GENERAL_LIGHTS	sporadic	outros spoofs possíveis

2. O que é o CAN Injection (sumarização do tema)

Contexto. O ataque foi tornado público por Ken Tindell (Canis Automotive Labs) a partir da investigação do roubo do Toyota RAV4 do pesquisador Ian Tabor. Os criminosos abriam o

para-choque, acessavam o conector do farol e, por meio de um dispositivo escondido em uma caixa de som Bluetooth (JBL), injetavam frames na rede CAN de controle. Com isso desativavam o imobilizador e destravavam as portas sem a chave. O caso recebeu o identificador **CVE-2023-29389** e dispositivos semelhantes são vendidos para dezenas de modelos de várias montadoras.

2.1 Mecanismo lógico: spoofing de frames clássicos

O núcleo do ataque é o **spoofing de frames do CAN Clássico**: o atacante transmite frames com o mesmo CAN-ID das mensagens legítimas da ECU da chave inteligente (smart key). A comunicação sem fio entre a chave e essa ECU é fortemente criptografada, mas as mensagens CAN que vão da smart key ECU para a ECU do motor (via gateway) e para a ECU das portas são **em texto claro e sem autenticação**. O gateway replica os frames forjados para o barramento de powertrain; a ECU do motor os aceita e desativa o imobilizador. Isso explora exatamente a **defesa de perímetro**: protege-se só a borda, supondo que nada alcança a parte interna.

2.2 Mecanismo físico/elétrico: o transceiver com *dominant-state-override*

A propriedade mais importante — e mais original — do CAN Injector é um **transceiver CAN modificado**. Normalmente o bit dominante (0) sempre vence e qualquer nó pode destruir um frame suspeito com um error frame. O transceiver do injetor faz o oposto: quando acionado, ele **dirige ativamente o estado recessivo**, mantendo as tensões de CAN-H/CAN-L abaixo do limiar do estado dominante. Com isso:

- **As outras ECUs não conseguem transmitir:** ao tentar impor dominante, leem recessivo (erro de bit), seguem as regras de erro e acabam em **Bus Off**. O injetor fica como único transmissor, sem colisões de arbitragem.
- **As outras ECUs não conseguem destruir o frame forjado** com error frames — o que **neutraliza o hardware anti-spoofing** baseado em error frames (p.ex. NXP *Stinger* e *CAN-HG* da Canis).
- **O ACK ainda funciona:** quando vários receptores impõem dominante juntos no bit de ACK, a combinação vence o override e todos confirmam o frame forjado.

Foram justamente os códigos de falha (DTCs) de perda de comunicação — gerados por esses erros — que permitiram a Ian Tabor rastrear o barramento CAN como vetor do roubo.

2.3 Por que o ataque é difícil de deter

Segundo Tindell, há duas defesas práticas: (1) **particionar** a rede com um gateway de segurança — que falha se o atacante tem acesso físico ao segmento confiável; ou (2) **proteção criptográfica dos frames**. Como o acesso físico é a premissa do ataque, resta essencialmente a criptografia: **AUTOSAR SecOC** (MAC + contador de frescor no payload) ou **CryptoCAN** (par de frames cifrado/autenticado), apoiados em um HSM **SHE** (emulável em software para atualizar ECUs em campo). Também se propõe detecção por **fingerprint temporal** (variações de 10-15 ns nos pulsos recessivos de cada nó) para inferir o nó de origem.

3. Reflexão 1 — O que é possível (e o que não é) reproduzir

A chave é distinguir as **duas camadas** do ataque. O YES, CARLA CAN usa barramentos CAN **virtuais** (`vcan0`/`vcan1` + `can-gw`, sobre `python-can` e DBC), o que define a fronteira do que se reproduz.

3.1 O que É possível reproduzir (o cerne lógico)

- **Spoofing de frames legítimos** — exatamente o que `attacks/reverse_engineering.py` já faz (mapeia funcionalidades para CAN-IDs e payloads). Como o cerne do CAN Injection é spoofing de frame clássico, essa essência é plenamente reproduzível (ver Seção 6).
- **A arquitetura de defesa de perímetro**: o par `vcan1` (atacante) em ponte com `vcan0` via gateway modela o encadeamento conector do farol → gateway → ECU do motor.
- **O efeito de ponta a ponta**: sem autenticação, os consumidores aceitam os frames forjados — portas destravam, freio de mão aciona, luzes mudam.
- **Ataques e detecção associados**: DoS por inundação e os IDS presentes (estatístico por tempo e ML por Isolation Forest).

3.2 O que NÃO é possível reproduzir (e por quê)

Não se reproduz a propriedade mais característica do ataque real: o **transceiver com dominant-state-override**. O `vcan` é uma abstração puramente de software da camada de enlace — **não existe camada física** (sem tensões CAN-H/CAN-L, sem arbitragem bit a bit, sem error frames ou Bus Off elétricos). Logo, não se reproduz:

- **O silenciamento das ECUs legítimas** (Bus Off): no simulador a ECU legítima continua transmitindo.
- **A supressão de error frames e a derrota do hardware anti-spoofing** (Stinger, CAN-HG).
- **O ACK-slot override** e o comportamento de arbitragem/colisão em geral.
- **Defesas de fingerprint físico** (jitter de 10-15 ns por nó): o `vcan` não tem variabilidade analógica por nó.
- **Contramedidas de hardware** (HSM SHE, transceivers seguros) — só emuláveis em software.

Síntese: o YES, CARLA CAN reproduz fielmente o **cerne lógico** do CAN Injection e seu **efeito** observável; não reproduz a **inovação de camada física** que torna o ataque real robusto contra defesas baseadas em error frames.

4. Reflexão 2 — O que precisaria ser acrescentado à arquitetura

Para investigar o CAN Injection de forma mais completa, seriam necessárias duas direções: tornar explícitos os elementos de confiança explorados e aproximar-se da camada física ausente.

- **Linha de base defendida para ser atacada.** Hoje tudo trafega em texto claro — não há o que burlar. Adicionar uma **ECU de imobilizador/autenticação** e uma mensagem de smart key com MAC (estilo SecOC).
- **Modelagem explícita de ECUs e gateway de segurança**, com **segmentos confiável x não-confiável** e filtragem de IDs no gateway.
- **Simulador de camada física / nível de bit** (ou co-simulação) que modele arbitragem, error frames e Bus Off — permitindo emular comportamentalmente o override e o silenciamento de ECUs.
- **Modelo de temporização/jitter por nó**, para viabilizar IDS por fingerprint temporal.
- **Atacante mais rico**: capaz de mirar uma ECU específica e vencer arbitragem de forma determinística.
- **Instrumentação/logging** em nível de frame e de erro.

5. Reflexão 3 — Mecanismos de defesa para o YES, CARLA CAN

As defesas organizam-se em três camadas: autenticação, detecção e segmentação.

5.1 Autenticação de mensagens (defesa principal)

- **AUTOSAR SecOC** sobre o CAN Clássico: anexar a cada mensagem crítica um **MAC truncado** + **contador de frescor**. Frames sem MAC válido são rejeitados. (Demonstrado na Seção 6, Fase 4.)
- **Esquema estilo CryptoCAN** (par de frames cifrado+autenticado) como alternativa mais forte.
- **Anti-replay** por contadores monotônicos/timestamps.

5.2 Detecção de intrusão (IDS)

- **IDS por temporização** (já existente): sinaliza IDs desconhecidos e desvios na cadência de mensagens periódicas — eficaz contra injeção que altera a frequência (ver Fase 3).
- **IDS por aprendizado de máquina** (Isolation Forest, já presente).
- **Detecção de duplicidade/colisão de origem**: dois transmissores do mesmo ID (o legítimo e o injetor).

5.3 Segmentação e endurecimento do gateway

- **Gateway de segurança** com **listas de permissão/bloqueio** de CAN-IDs (inspirado no Stinger).
- **Rate limiting** por ID, mitigando injeção em rajada e DoS.

6. Experimento reproduzível (demonstração)

Foi implementada no repositório uma demonstração autocontida do CAN Injection: `attacks/can_injection_demo.py`. Ela reaproveita os artefatos reais do projeto — os payloads de `attacks/reverse_engineering.py`, o schema `data/carla.dbc` e o IDS de `defense/id_time_intrusion_detection.py` — encenando, em um único processo, ECUs legítimas, uma ECU-vítima, o IDS e um atacante.

6.1 Como executar

Modo autocontido (não precisa de `sudo` nem do kernel vcan — ideal para estudar e apresentar):

```
$ python -m attacks.can_injection_demo
```

Usando a rede CAN virtual real do projeto (após `./1_up_environment.sh`, com `vcan0`/`vcan1` ativos):

```
$ python -m attacks.can_injection_demo --iface socketcan
```

6.2 Fases e resultados

Fase	Ação	Resultado observado
1	Tráfego normal das ECUs legítimas	Porta trancada; IDS em baseline (apenas falsos positivos por jitter de software).
2	Spoof de DOORS (0x60C), payload [0x01]	**Porta destranca sem autorização.** O IDS por tempo **não detecta** (ID legítimo, tipo sporadic) → bypass.

3	Flood de HAND_BRAKE (0x604) a ~50 msg/s	Freio de mão acionado por injeção; o IDS **detecta** a anomalia de cadência do ID periódico.
4	Defesa SecOC/MAC ativada na porta	O mesmo spoof é **rejeitado** (MAC inválido); só o comando autenticado destrava.

6.3 Trecho da saída real

```
FASE 1  -  Trafego normal (sem ataque)
[IDS]    baseline: 50 frames legitimos; 14 falsos positivos (jitter)

FASE 2  -  CAN INJECTION: spoof de DOORS (0x60C)
[ATAQUE] injetando 0x60C payload=[0x01]
[VITIMA] >>> PORTA DESTRANCADA sem autorizacao! (frame 0x60C aceito)
[IDS]    anomalias no 0x60C = 0   ->  BYPASS

FASE 3  -  CAN INJECTION: flood de HAND_BRAKE (0x604, periodico)
[ATAQUE] injetando 0x604 a ~50 msg/s sobre a cadencia de 0.2s
[VITIMA] freio de mao ACIONADO por frame injetado
[IDS]    anomalias no 0x604 = 158 ->  DETECTADO

FASE 4  -  DEFESA SecOC/MAC
[ATAQUE] repete o mesmo spoof 0x60C=[0x01] (sem a chave)
[VITIMA] comando de porta REJEITADO (MAC invalido -> spoof bloqueado)
[INFO]   apos 10 tentativas de spoof: porta = TRANCADA (nenhuma aceita)
[VITIMA] porta destrancada (comando AUTENTICADO aceito)
```

6.4 Interpretação

A demonstração conecta as três reflexões: a **Fase 2** mostra o cerne lógico reproduzível e, de quebra, um **limite do IDS atual** (um spoof com ID legítimo e esporádico passa despercebido); a **Fase 3** mostra a **detecção** de injeção que altera a cadência de um ID periódico; e a **Fase 4** implementa a **defesa por autenticação** (MAC estilo SecOC) que o ataque não consegue contornar sem a chave — exatamente a defesa que Tindell aponta como a única eficaz quando há acesso físico. Permanece fora do alcance, por falta de camada física no `vcan`, a reprodução do override de estado dominante (Seção 3).

Referências

- TINDELL, K. *CAN Injection: keyless car theft.* Canis Automotive Labs, 03/04/2023. kentindell.github.io/2023/04/03/can-injection/
- TINDELL, K. *The CAN Injection attack.* CAN Newsletter 2/2023, CiA (CAN in Automation), p. 20-22.
- CVE-2023-29389 — CAN Injection vulnerability (Toyota/Lexus e outros).
- AUTOSAR. *Specification of Secure Onboard Communication (SecOC).*
- Canis Automotive Labs. *CryptoCAN* e *CAN-HG*. canislabs.com
- BOSCH. *CAN Specification 2.0* (ISO 11898).